



CICLO FORMATIVO DE GRADO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

PROYECTO FIN DE CICLO

Título: Omnia Manager

Alumno: Susana C. Bitar A.

N.º Alumno: 356486

DECLARACIÓN JURADA DE AUTORÍA

D. / Dña. Susana Coromoto Bitar Azevedo, con N.I.E. Y6119116A, como autor/es de este documento académico, titulado:

Omnia Manager,

y presentado como Proyecto Fin de Ciclo para la obtención del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW),

DECLARO QUE

Soy/somos el/los único/s autor/es del trabajo, con la excepción de referencias a contenidos o ideas de otros autores, en cuyo caso han sido explícitamente citados.

El trabajo remitido es un documento original, que no ha sido publicado, ya sea total o parcialmente, ni presentado para obtención de un título académico en ninguna institución académica u organización.

No he trasgredido ninguna norma universitaria con respecto al plagio ni a las leyes establecidas que protegen la propiedad intelectual.

Soy consciente de que el hecho de no respetar estos extremos es una falta grave de integridad académica y podrá ser objeto de sanciones.

En Madrid, a 22 de mayo de 2026

(firma)

(firma)

(firma)

Fdo.:

Fdo.:

Fdo.:

RESUMEN

Omnia Manager es una aplicación web corporativa creada para solucionar un problema muy común en las empresas: la desorganización de los datos internos. Su función principal es centralizar la gestión de los empleados, el control de los proyectos activos y las asignaciones de personal en una única herramienta.

A nivel técnico, la aplicación está desarrollada como un sistema Full-Stack. El backend está programado en Java 17 con Spring Boot para construir una API REST, mientras que el frontend se ha diseñado como una Single Page Application (SPA) utilizando Vue 3 y Vuetify. Todos los datos se almacenan en una base de datos MySQL. Para garantizar la seguridad del sistema, las contraseñas se cifran mediante BCrypt, el control de sesiones se gestiona en el frontend a través de Pinia, y se ha implementado un sistema de bajas lógicas para no perder nunca el historial de la empresa. Por último, todo el entorno está paquetizado utilizando Docker.

ABSTRACT

Omnia Manager is a corporate web application created to solve a very common problem in companies: disorganized internal data. Its main function is to centralize employee management, project tracking, and staff assignments into a single tool.

On a technical level, the application is developed as a Full-Stack system. The backend is programmed in Java 17 using Spring Boot to build a REST API, while the frontend is designed as a Single Page Application (SPA) using Vue 3 and Vuetify. All data is stored in a MySQL database. To guarantee system security, passwords are encrypted using BCrypt, session control is managed on the frontend via Pinia, and a soft-delete system has been implemented to preserve the company's historical records. Finally, the entire environment is containerized using Docker.

CONTENIDO

1. INTRODUCCIÓN.....	7
2. OBJETIVOS.....	8
2.1. Objetivo general	8
2.2. Objetivos específicos	8
3. REQUISITOS FUNCIONALES.....	9
3.1. Autenticación y acceso	9
3.2. Panel de control (Dashboard)	9
3.3. Gestión de empleados	9
3.4. Gestión de proyectos.....	10
3.5. Asignaciones	10
3.6. Analítica	10
3.7. Perfil de usuario.....	10
3.8. Protección de datos (Patrón DTO).....	11
4. DISEÑO DE LA SOLUCIÓN.....	12
4.1. Mapa de navegación.....	12
4.2. Modelo de base de datos	12
4.3. Diseño de la arquitectura	16
4.4. Lenguajes y tecnologías utilizados.....	18
5. DISEÑO TÉCNICO.....	20
5.1. Libro de estilo	20
5.2. Definición de componentes.....	28
5.3. Diseño de tablas de la base de datos.....	29
6. DEMOSTRACIÓN	32
7. CONCLUSIONES.....	33
8. BIBLIOGRAFÍA.....	34
9. ANEXOS.....	36
9.1. Código fuente	36

ÍNDICE DE ILUSTRACIONES

<i>Ilustración 1: mapa de navegación Omnia Manager. Fuente: elaboración propia.</i>	<i>12</i>
<i>Ilustración 2: modelo Entidad/Relación Omnia Manager. Fuente: elaboración propia.</i>	<i>13</i>
<i>Ilustración 3: diagrama de arquitectura Omnia Manager. Fuente: elaboración propia.....</i>	<i>16</i>
<i>Ilustración 4: logotipo Omnia Manager, versión oscura y blanca. Fuente: elaboración propia.</i>	<i>20</i>
<i>Ilustración 6: favicon Omnia Manager. Fuente: elaboración propia.</i>	<i>21</i>
<i>Ilustración 7: jerarquía tipográfica Omnia Manager. Fuente: elaboración propia.....</i>	<i>21</i>
<i>Ilustración 8: paleta de colores Omnia Manager. Fuente: elaboración propia.</i>	<i>22</i>
<i>Ilustración 9: vista Dashboard — tarjetas StatCard y CrystalCard. Fuente: elaboración propia.....</i>	<i>23</i>
<i>Ilustración 10: variantes del componente FButton. Fuente: elaboración propia.....</i>	<i>24</i>
<i>Ilustración 11: estados del campo de formulario CrystalInput. Fuente: elaboración propia.....</i>	<i>25</i>
<i>Ilustración 12: barra de búsqueda EliteSearch en reposo. Fuente: elaboración propia.....</i>	<i>25</i>
<i>Ilustración 13: barra de búsqueda EliteSearch con foco. Fuente: elaboración propia.....</i>	<i>26</i>
<i>Ilustración 14: notificaciones CrystalToast de éxito. Fuente: elaboración propia.....</i>	<i>26</i>
<i>Ilustración 15: notificaciones CrystalToast de error. Fuente: elaboración propia.</i>	<i>27</i>
<i>Ilustración 16: lista de proyectos con fila en hover. Fuente: elaboración propia.....</i>	<i>27</i>

ÍNDICE DE TABLAS

<i>Tabla 1: Estructura de la tabla US_USUARIOS. Fuente: elaboración propia.....</i>	<i>30</i>
<i>Tabla 2: Estructura de la tabla EM_EMPLEADOS. Fuente: elaboración propia.</i>	<i>30</i>
<i>Tabla 3: Estructura de la tabla PR_PROYECTOS. Fuente: elaboración propia.</i>	<i>31</i>
<i>Tabla 4: Estructura de la tabla PR_EMPLEADOS_PROYECTO. Fuente: elaboración propia.</i>	<i>31</i>
<i>Tabla 5: Credenciales de acceso para la demo. Fuente: elaboración propia.</i>	<i>32</i>
<i>Tabla 6: Ficheros principales del proyecto Omnia Manager. Fuente: elaboración propia.</i>	<i>36</i>

1. INTRODUCCIÓN

Cuando trabajas en cualquier empresa, te das cuenta rápido de un problema: los datos de los empleados y los proyectos suelen estar repartidos por todas partes. Unas cosas están en Excel, otras en correos, y al final, buscar a qué proyecto está asignado alguien o revisar quién sigue activo en la plantilla supone perder mucho tiempo.

A raíz de ver esto, decidí crear Omnia Manager. Quería programar una herramienta interna que sirviera para tener a toda la plantilla y todos los proyectos controlados desde un único sitio, sin complicaciones y de forma segura.

Técnicamente, este proyecto me ha servido para poner en práctica todo el ciclo de desarrollo de una aplicación web real. He programado desde cero la base de datos, el backend completo con Spring Boot y el frontend con Vue 3, además de montarlo todo con contenedores Docker para que funcione a la primera en cualquier equipo.

El resultado es una plataforma donde un administrador puede dar de alta empleados, gestionar proyectos, asignar a la gente donde haga falta y, sobre todo, ver un panel de control rápido para saber en qué estado está la empresa al instante.

2. OBJETIVOS

Para guiar el desarrollo de Omnia Manager, se han definido una serie de objetivos centrados en resolver los problemas descritos y aportar un valor real a la operativa de la empresa.

2.1. Objetivo general

Desarrollar una aplicación web completa (Full-Stack) orientada a gestionar la plantilla y los proyectos de una empresa, asegurando que la información esté centralizada, los datos sean seguros y el historial de los empleados nunca se pierda.

2.2. Objetivos específicos

Para alcanzar el objetivo general, el proyecto se ha dividido en los siguientes objetivos técnicos y funcionales:

- **Crear una interfaz rápida e intuitiva:** Programar el frontend como una Single Page Application (SPA) para que la navegación sea instantánea y el diseño minimalista facilite el trabajo diario.
- **Mantener el historial de la empresa:** Implementar bajas lógicas (soft deletes) en la base de datos. De esta forma, cuando un empleado o un proyecto se da de baja, no se borra físicamente y la empresa conserva su registro histórico.
- **Bloquear el acceso público:** Configurar la seguridad del sistema para que no exista registro libre. Solo se puede acceder con credenciales validadas contra la base de datos y cifradas.
- **Mostrar métricas en tiempo real:** Desarrollar módulos de analítica y un panel de control principal que procesen los datos del servidor al instante, mostrando estadísticas como la rotación de plantilla o el volumen de proyectos activos.
- **Separar la base de datos del cliente:** Programar el backend utilizando el patrón DTO. Con esto, el servidor solo envía al navegador la información estrictamente necesaria, protegiendo la estructura real de la base de datos.
- **Simplificar la instalación:** Configurar Docker y Docker Compose para empaquetar toda la aplicación (base de datos, backend y frontend), logrando que cualquier persona pueda arrancarla con un solo comando.

3. REQUISITOS FUNCIONALES

Omnia Manager sirve para centralizar en un único sitio todo lo relacionado con empleados, proyectos y asignaciones. A continuación, se detallan las capacidades principales de la aplicación.

3.1. Autenticación y acceso

La aplicación está completamente protegida a nivel de acceso. Para entrar, el administrador introduce su correo y contraseña en la pantalla de Login. El backend comprueba las credenciales contra la base de datos, donde las contraseñas están cifradas con BCrypt. Si son correctas, el frontend guarda el estado de la sesión en el navegador (mediante Pinia y localStorage) y redirige al usuario al panel principal. Mientras la sesión está activa, el usuario puede navegar por toda la parte privada de la aplicación.

La creación de nuevas cuentas de administrador y la recuperación de contraseñas no se hacen desde la web, sino directamente a través de los endpoints de la API. Esto es una decisión de diseño tomada para evitar que cualquier persona pueda registrarse libremente en el sistema, ya que se trata de una herramienta de uso interno corporativo.

3.2. Panel de control (Dashboard)

Al iniciar sesión, la primera pantalla es el Dashboard. Está diseñado para funcionar como una página de inicio rápida. Muestra un saludo, contadores con la información más importante (cantidad de personal activo, proyectos en marcha) y alertas sobre proyectos que necesitan atención. Desde aquí también existen accesos directos para dar de alta a empleados o proyectos rápidamente.

3.3. Gestión de empleados

Desde el módulo de empleados, el administrador puede consultar el listado completo de la plantilla, añadir nuevos trabajadores, editar sus datos o darlos de baja. Al crear un empleado hay que rellenar campos como el NIF, nombre completo, apellidos, categoría profesional y nivel de estudios. El formulario valida el formato del NIF antes de enviarlo al servidor, por lo que si está mal escrito no deja guardar.

Cuando se da de baja a un empleado, el sistema no borra el registro de la base de datos. En su lugar, anota la fecha en la que se produjo la baja (soft delete). Esto es importante porque permite mantener el historial completo de la empresa sin perder datos.

3.4. Gestión de proyectos

El módulo de proyectos permite gestionar todo el portfolio activo de la organización. Se puede crear un nuevo proyecto indicando su nombre, la ciudad donde se realiza, las fechas de inicio y fin previstas, y su estado (activo, pausado o finalizado). También se pueden editar los datos de un proyecto existente o darlo de baja.

Al igual que con los empleados, el borrado es lógico. Además, si un proyecto tiene empleados asignados en ese momento, el sistema no deja darlo de baja y muestra un mensaje de error explicando que primero hay que retirar las asignaciones.

3.5. Asignaciones

Este módulo conecta los empleados con los proyectos. El administrador puede vincular a un trabajador a un proyecto concreto, registrándose en el sistema la fecha de inicio de la asignación. El control operativo se realiza entrando al detalle de cada proyecto, donde el usuario puede visualizar exactamente qué empleados están trabajando en él actualmente. Cuando un empleado termina su participación en dicho proyecto, el administrador puede desvincularlo fácilmente eliminando la asignación de la base de datos.

3.6. Analítica

A diferencia del Dashboard, que ofrece una visión operativa inmediata, el módulo de Analítica está diseñado para la inteligencia de negocio. Esta sección procesa el histórico completo de la base de datos para renderizar gráficos interactivos complejos (anillos, barras y líneas de evolución). Permite analizar métricas profundas como el índice de rotación (comparando empleados activos frente a bajas históricas), la carga de los proyectos principales, la distribución geográfica de las sedes y la evolución de las contrataciones por años. Todos estos cálculos se resuelven dinámicamente en el servidor para evitar sobrecargar el navegador del cliente.

3.7. Perfil de usuario

El sistema cuenta con un área de configuración ("Perfil") orientada a la personalización de la experiencia del administrador. Desde aquí se puede actualizar el nombre a mostrar y gestionar una fotografía personalizada (avatar). Un detalle técnico importante es que el correo electrónico permanece bloqueado y en modo de solo lectura, ya que actúa como el identificador inmutable de la cuenta corporativa. Los cambios realizados aquí se propagan en tiempo real al resto de la interfaz gracias al gestor de estado global.

3.8. Protección de datos (Patrón DTO)

Para garantizar la seguridad de la información, la arquitectura del backend implementa de forma estricta el patrón DTO (Data Transfer Object). Esto significa que las entidades reales de la base de datos (JPA) nunca se exponen al cliente. Antes de enviar cualquier respuesta HTTP, el servidor mapea la información, descartando datos sensibles o campos internos que no deben ser públicos. De esta forma, se protege la estructura interna del modelo de datos frente a posibles análisis de vulnerabilidades desde el frontend.

4. DISEÑO DE LA SOLUCIÓN

Antes de escribir una sola línea de código, se definió la estructura de la aplicación: el flujo de navegación entre vistas y el modelo de datos de la base de datos.

4.1. Mapa de navegación

La aplicación sigue el modelo de Single Page Application (SPA), lo que significa que todas las transiciones entre pantallas ocurren sin recargas de página. Esto obliga a planificar bien las rutas y los puntos de acceso desde el primer momento. El siguiente diagrama refleja la estructura de navegación tal y como la experimenta el usuario, distinguiendo entre la zona pública (accesible sin autenticación) y la zona privada (protegida tras el Login), donde un menú lateral permanente permite moverse entre los distintos módulos del sistema.

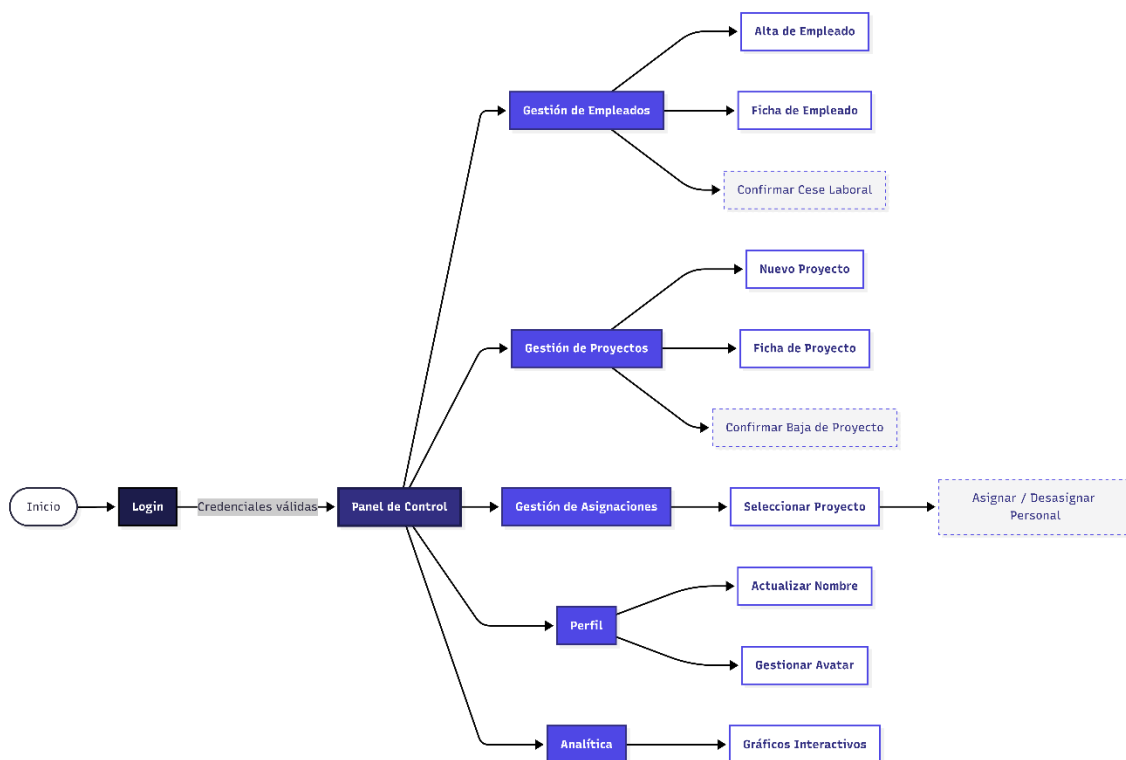


Ilustración 1: mapa de navegación Omnia Manager. Fuente: elaboración propia.

4.2. Modelo de base de datos

El modelo de datos de Omnia se compone de cuatro tablas que cubren toda la funcionalidad de la aplicación: la autenticación de usuarios, la gestión de empleados, la gestión de proyectos y la asignación de empleados a proyectos. A continuación, se presenta el diagrama Entidad/Relación junto con la descripción de cada entidad y sus relaciones.

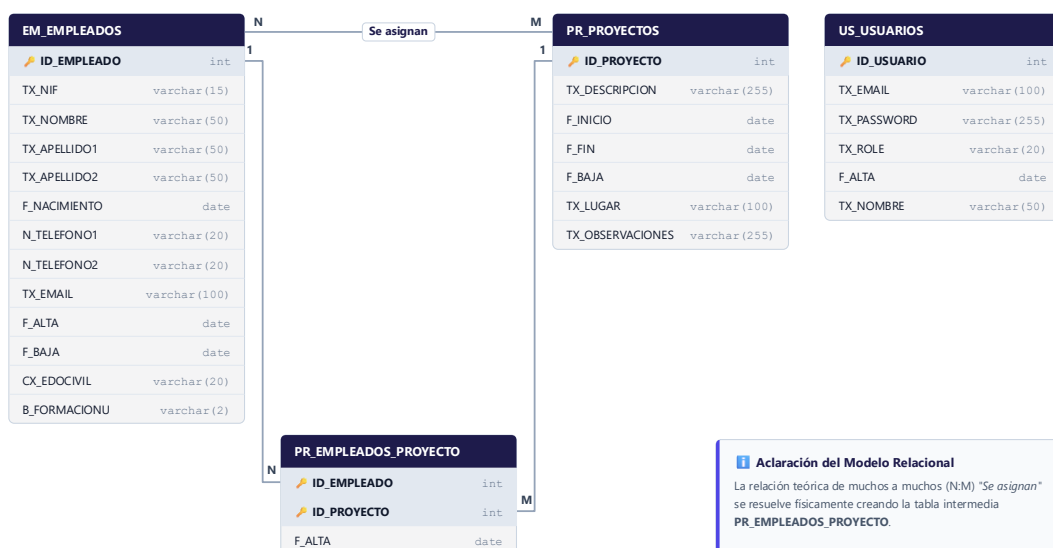


Ilustración 2: modelo Entidad/Relación Omnia Manager. Fuente: elaboración propia.

Descripción de entidades y atributos

- **US_USUARIOS**

- **Descripción:** almacena las credenciales de acceso al sistema.
- **Atributos**
 - **ID_USUARIO (PK):** identificador numérico autoincremental.
 - **TX_EMAIL (UK, NOT NULL):** correo corporativo. Actúa como identificador inmutable de la cuenta y no puede modificarse una vez creada.
 - **TX_PASSWORD (NOT NULL):** contraseña cifrada con BCrypt.
 - **TX_ROLE (NOT NULL):** rol del usuario en el sistema.
 - **F_ALTA (NOT NULL):** fecha de creación de la cuenta.
 - **TX_NOMBRE:** nombre a mostrar en la interfaz, configurable desde el área de perfil.

- **EM_EMPLEADOS**

- **Descripción:** contiene la ficha de cada trabajador. La baja en la empresa no elimina el registro: se rellena el campo F_BAJA con la fecha de cese para conservar el historial completo (*soft delete*).
- **Atributos**
 - **ID_EMPLEADO (PK):** identificador numérico autoincremental.
 - **TX_NIF (UK, NOT NULL):** número de identificación fiscal, de 9 caracteres.
 - **TX_NOMBRE (NOT NULL):** nombre del trabajador.
 - **TX_APELLIDO1 (NOT NULL):** primer apellido.
 - **TX_APELLIDO2 (NOT NULL):** segundo apellido.
 - **F_NACIMIENTO (NOT NULL):** fecha de nacimiento.
 - **N_TELEFONO1 (NOT NULL):** teléfono principal.
 - **N_TELEFONO2 (NOT NULL):** teléfono secundario.
 - **TX_EMAIL (NOT NULL):** correo del trabajador.
 - **F_ALTA (NOT NULL):** fecha de incorporación a la empresa.
 - **F_BAJA:** fecha de cese. Si es nula, el empleado está activo.
 - **CX_EDOCIVIL (NOT NULL):** estado civil, un carácter (S soltero, C casado).
 - **B_FORMACIONU (NOT NULL):** indica si el empleado tiene formación universitaria (S sí, N no).

- **PR_PROYECTOS**

- **Descripción:** almacena la información de cada proyecto de la compañía. Al igual que los empleados, los proyectos no se eliminan físicamente; se registra una fecha de baja lógica en F_BAJA cuando finalizan o se cancelan.

- **Atributos**
 - **ID_PROYECTO (PK):** identificador numérico autoincremental.
 - **TX_DESCRIPCION (NOT NULL):** descripción del proyecto, hasta 125 caracteres.
 - **F_INICIO (NOT NULL):** fecha de inicio del proyecto.
 - **F_FIN:** fecha prevista de finalización.
 - **F_BAJA:** fecha de cierre lógico. Si es nula, el proyecto está vigente.
 - **TX_LUGAR (NOT NULL):** sede o localización del proyecto.
 - **TX_OBSERVACIONES:** notas adicionales, hasta 300 caracteres.
- **PR_EMPLEADOS_PROYECTO**
 - **Descripción:** tabla de unión que resuelve la relación muchos a muchos entre empleados y proyectos. Registra cuándo se incorporó cada empleado a cada proyecto y no contiene datos adicionales más allá de los identificadores y la fecha de asignación.
 - **Atributos**
 - **ID_PROYECTO (PK/FK):** parte de la clave primaria compuesta; referencia a PR_PROYECTOS.
 - **ID_EMPLEADO (PK/FK):** parte de la clave primaria compuesta; referencia a EM_EMPLEADOS.
 - **F_ALTA (NOT NULL):** fecha en que el empleado fue incorporado al proyecto.

Relaciones y cardinalidades

La relación entre empleados y proyectos se resuelve en la base de datos mediante la entidad de unión PR_EMPLEADOS_PROYECTO, que materializa la cardinalidad M:N entre EM_EMPLEADOS y PR_PROYECTOS. Su clave primaria es compuesta: los campos ID_EMPLEADO e ID_PROYECTO, ambos clave foránea hacia sus respectivas tablas padre. El único atributo propio de esta entidad

es F_ALTA, que registra la fecha en que el empleado fue incorporado al proyecto. La tabla US_USUARIOS no mantiene relaciones de clave foránea con el resto del esquema de forma deliberada: almacena exclusivamente las cuentas de administrador del sistema, que autentican y operan la aplicación, pero no se modelan como empleados ni participan en la gestión de proyectos. Esta separación permite controlar el acceso a la plataforma de forma independiente al ciclo de vida de los recursos humanos.

- **EM_EMPLEADOS — PR_PROYECTOS (M:N)**

- **Descripción:** un empleado puede estar asignado a varios proyectos y un proyecto puede tener varios empleados. La relación se resuelve mediante la tabla PR_EMPLEADOS_PROYECTO.

4.3. Diseño de la arquitectura

Omnia Manager sigue el modelo cliente-servidor. Los tres componentes principales —frontend, backend y base de datos— se ejecutan en contenedores Docker independientes, orquestados mediante un único fichero docker-compose.yml.

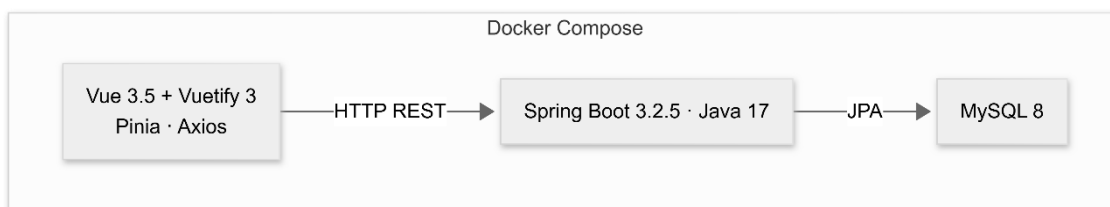


Ilustración 3: diagrama de arquitectura Omnia Manager. Fuente: elaboración propia.

Backend

El backend de Omnia Manager está desarrollado con Spring Boot 3.2.5 sobre Java 17, e implementa el patrón en tres capas (Controller → Service → Repository). Los principales servicios programados son:

- **Autenticación:** Login con validación de credenciales y cifrado de contraseñas con BCrypt.
- **Gestión de empleados:** CRUD completo con borrado lógico (*soft delete*) mediante el campo F_BAJA.

- **Gestión de proyectos:** CRUD completo con borrado lógico y restricción de baja si el proyecto tiene asignaciones activas.
- **Asignaciones:** gestión de la relación entre empleados y proyectos, incluyendo alta y baja de asignaciones.
- **Analítica:** cálculos agregados en el servidor para generar las métricas del módulo de analítica sin sobrecargar el navegador.
- **Perfil de usuario:** actualización de nombre y avatar. El correo electrónico permanece inmutable al actuar como identificador de cuenta.

Frontend

El frontend es una Single Page Application (SPA) desarrollada con Vue 3.5 y Vuetify 3. La estructura de componentes incluye:

- **Layouts:** plantilla pública (pantalla de Login) y MainLayout (zona privada con menú lateral y barra superior).
- **Componentes reutilizables:** CrystalCard, CrystalInput, FButton y StatCard, utilizados de forma consistente en todas las vistas.
- **Vue Router 4:** enrutamiento interno de la SPA con guardas de navegación que redirigen a /login si no hay sesión activa.
- **Pinia:** gestor de estado global que almacena los datos de la sesión y los persiste en el localStorage del navegador.
- **Axios:** cliente HTTP para realizar las peticiones a la API REST del backend.
- **Chart.js 4.5:** renderizado de gráficos interactivos (anillos, barras y líneas) en el módulo de Analítica.

Servidor de aplicaciones

Spring Boot incluye un servidor Tomcat embebido, por lo que no se requiere ningún servidor de aplicaciones externo. La API REST arranca directamente al lanzar el contenedor del backend.

Servidor web

Nginx Alpine actúa como servidor web del frontend. Durante la construcción del contenedor, Node 22 genera el *bundle* estático de Vue y se copia en Nginx, que lo sirve en el puerto 80.

Gestor de base de datos

MySQL 8 es el sistema gestor de base de datos relacional. Almacena toda la información persistente de la aplicación en cuatro tablas (US_USUARIOS, EM_EMPLEADOS, PR_PROYECTOS, PR_EMPLEADOS_PROYECTO). La conexión se establece desde el contenedor del backend mediante JDBC, configurada a través de las variables de entorno del docker-compose.yml. Al arrancar, MySQL inicializa el esquema automáticamente con el script DDL del proyecto.

Contenedores Docker

La aplicación se despliega con Docker Compose, orquestando los tres servicios en contenedores aislados que se comunican a través de una red interna:

- **omnia-frontend:** contenedor Nginx Alpine que sirve el bundle estático de Vue 3.5.
- **omnia-backend:** contenedor con la API REST de Spring Boot 3.2.5.
- **omnia-db:** contenedor MySQL 8 con volumen persistente para los datos.
- **Red interna:** los contenedores se comunican entre sí mediante la red privada de Docker Compose.

4.4. Lenguajes y tecnologías utilizados

A continuación, se recogen los lenguajes de programación, frameworks y herramientas principales empleados en el desarrollo de Omnia Manager.

Frontend

- **JavaScript (ES2022) y Vue 3.5:** lenguaje y framework principal del cliente. Vue se usa con la Composition API y el sistema de Single File Components (.vue).
- **Vuetify 3:** biblioteca de componentes UI basada en Material Design, que proporciona la estructura visual de la aplicación: formularios, tablas, tarjetas y navegación lateral.

- **Pinia:** gestor de estado global para mantener y compartir entre vistas los datos de sesión del usuario.
- **Chart.js 4.5:** biblioteca de gráficos interactivos utilizada en el módulo de Analítica para renderizar anillos, barras y líneas de evolución.
- **Axios:** cliente HTTP para realizar las peticiones a la API REST desde el navegador.
- **Vue Router 4:** enrutador oficial de Vue, responsable de la navegación entre las vistas de la SPA sin recargar la página.

Backend

- **Java 17:** lenguaje de programación del servidor.
- **Spring Boot 3.2.5:** framework que estructura el backend en capas (Controller, Service, Repository) y arranca el servidor embebido Tomcat.
- **Spring Data JPA + Hibernate:** capa de acceso a datos que mapea las entidades Java a las tablas MySQL mediante anotaciones JPA.
- **BCrypt:** algoritmo de cifrado unidireccional usado para almacenar las contraseñas de los usuarios de forma segura.

Base de datos

- **MySQL 8:** sistema gestor de bases de datos relacional. El esquema se versiona mediante un script DDL incluido en el repositorio.

Infraestructura y herramientas

- **Docker y Docker Compose:** el frontend, el backend y la base de datos se ejecutan en contenedores independientes orquestados por un único fichero docker-compose.yml.
- **Maven:** gestor de dependencias y herramienta de construcción del proyecto Spring Boot.
- **npm:** gestor de paquetes del proyecto Vue.

5. DISEÑO TÉCNICO

Los siguientes apartados describen en detalle cómo se ha construido la solución: la guía visual que define la estética de la interfaz, los componentes técnicos clave y el modelo relacional de la base de datos.

5.1. Libro de estilo

El diseño visual de Omnia Manager sigue la estética *Fine-Line Minimalist*: superficies blancas, bordes muy finos, sombras sutiles y un uso deliberado del espacio en blanco para reducir la fatiga visual. El resultado es una interfaz que prioriza la legibilidad de los datos sobre la ornamentación.

Logotipo

El logotipo de Omnia Manager se presenta en dos versiones:

- **Versión oscura:** símbolo en índigo corporativo (#312E81) con texto en #1E1B4B. Se usa sobre fondos claros (blanco o gris perla).
- **Versión blanca:** símbolo y texto en #FFFFFF. Se usa sobre fondos oscuros, como la barra superior de la aplicación.

El símbolo es un nodo de conexión formado por tres elementos (anillo exterior, nodo superior y punto central), que representa la interconexión entre empleados, proyectos y la empresa. La tipografía del logotipo es **Outfit ExtraBold (800)**. Ambas versiones están en formato SVG para garantizar nitidez a cualquier escala.

El favicon replica el símbolo del logotipo en formato SVG y se adapta automáticamente al modo claro u oscuro del sistema operativo del usuario, sin necesidad de intervención manual.



Ilustración 4: logotipo Omnia Manager, versión oscura y blanca. Fuente: elaboración propia.

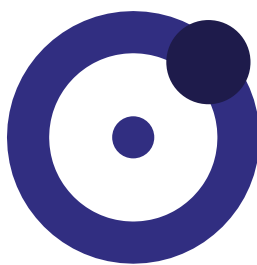


Ilustración 5: favicon Omnia Manager. Fuente: elaboración propia.

Tipografía

Se utiliza un sistema tipográfico dual que separa visualmente la marca de los datos:

- **Outfit (Google Fonts):** tipografía de display. Se usa en títulos de página, encabezados de sección, valores KPI grandes, títulos de tarjetas y el propio logotipo. Su geometría moderna y limpia aporta carácter a la interfaz.
- **Inter (Google Fonts):** tipografía de datos. Se usa en cuerpos de texto, tablas, campos de formulario, etiquetas y cualquier elemento que presente información numérica. Diseñada específicamente para pantallas, sus números tabulares evitan desalineaciones en columnas de datos.

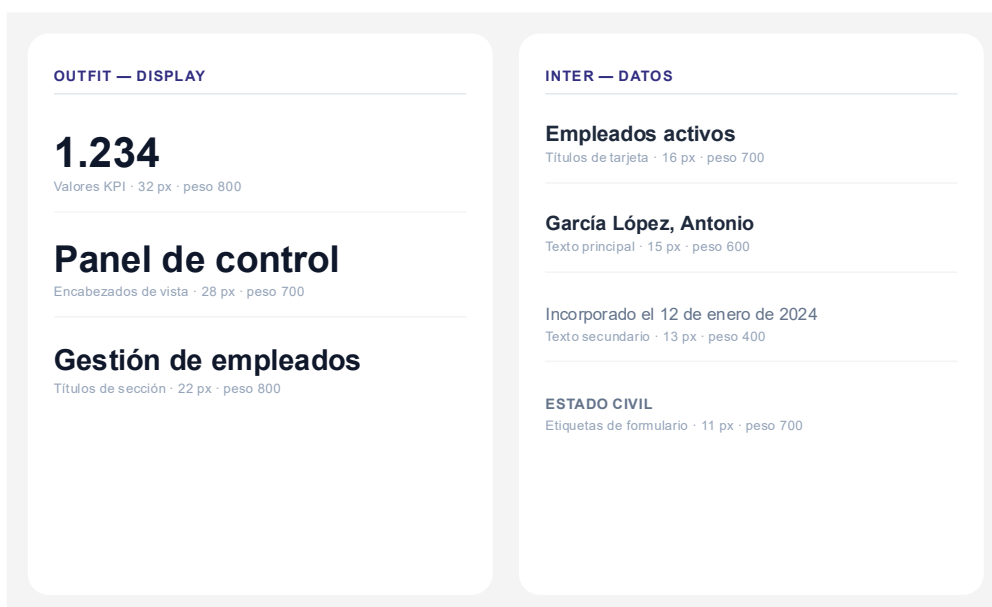


Ilustración 6: jerarquía tipográfica Omnia Manager. Fuente: elaboración propia.

Paleta de colores

La paleta se basa en la familia de índigos para transmitir confianza corporativa, complementada con un fondo gris perla que evita el blanco puro y reduce la fatiga visual en sesiones largas.

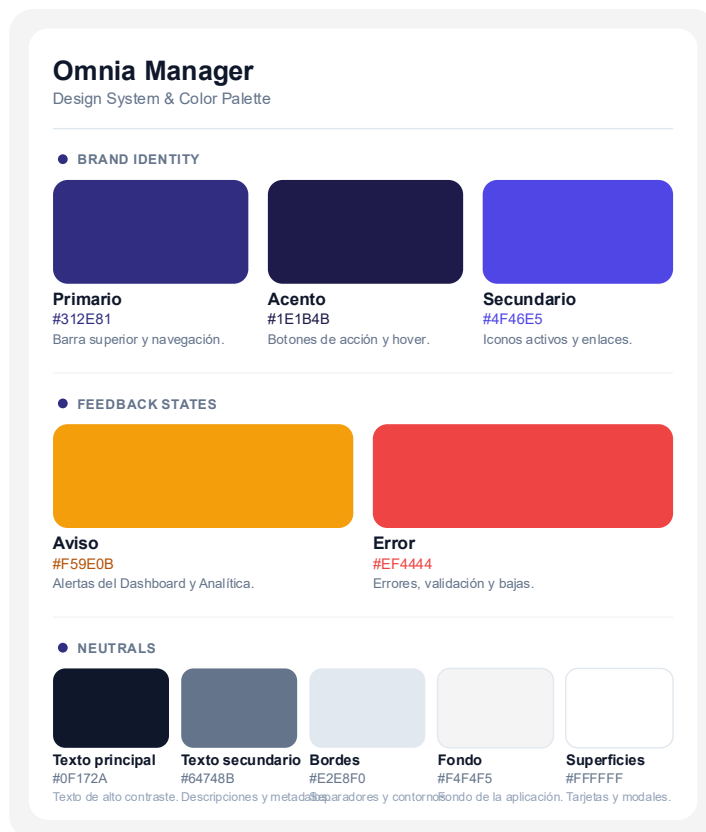


Ilustración 7: paleta de colores Omnia Manager. Fuente: elaboración propia.

Rejilla y espaciado

Cada vista de la aplicación se encuadra en un contenedor centrado de ancho máximo 1280 px, con un padding lateral de 32 px en escritorio que se reduce a 16 px en pantallas pequeñas. Los elementos internos se separan con un espacio de 24 px entre bloques para mantener la respiración visual.

El sistema de rejilla varía según el módulo:

- **Formularios:** cuadrícula de dos columnas de igual peso que distribuye los campos en parejas (nombre y apellido, teléfono 1 y teléfono 2, etc.). En pantallas menores de 768 px los campos se apilan en una sola columna para evitar que se compriman.

- **Dashboard:** *Bento Grid* con bloques de tamaños asimétricos, pensado para jerarquizar visualmente la información: los KPIs principales ocupan más espacio que los elementos secundarios.
- **Perfil:** dos columnas de ancho fijo la izquierda (área del avatar) y flexible la derecha (campos de edición), que colapsan a una sola columna en móvil.

Los paddings interiores de todos los contenedores se mantienen entre 20 px y 24 px para garantizar que el contenido no quede pegado a los bordes y la lectura sea cómoda.

Componentes visuales

Tarjetas (CrystalCard)

Las tarjetas son el contenedor visual principal de la interfaz. Usan fondo blanco con un ligero efecto de desenfoque (blur) que las separa sutilmente del fondo sin romper la continuidad visual. Los bordes son casi invisibles y las esquinas están muy redondeadas (24 px), lo que les da un aspecto moderno y suave.

La sombra base es intencionalmente discreta para no competir con el contenido que contiene la tarjeta. Al pasar el cursor, la tarjeta se eleva ligeramente y la sombra se abre de forma gradual, generando una sensación de profundidad sin resultar brusca.

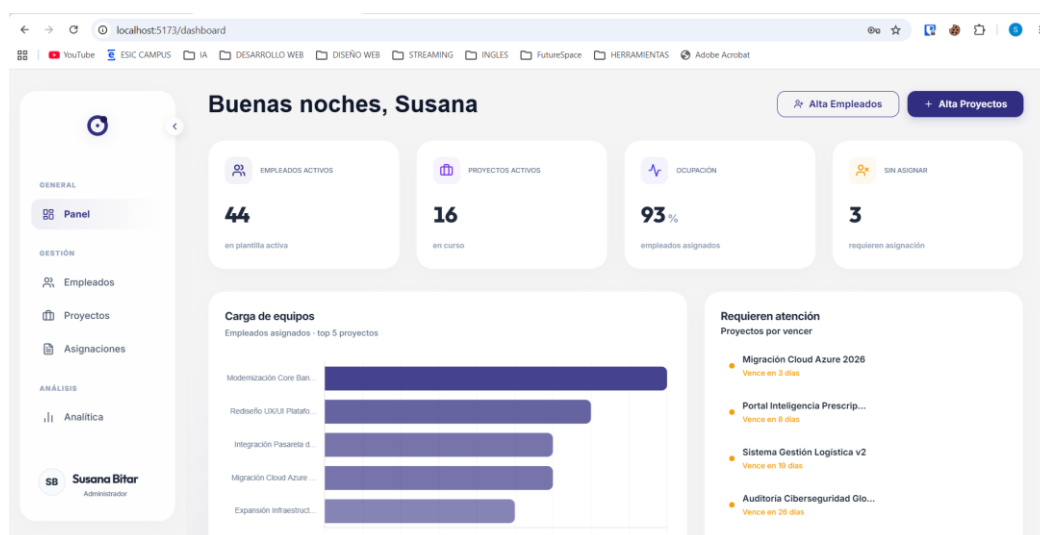


Ilustración 8: vista Dashboard — tarjetas StatCard y CrystalCard. Fuente: elaboración propia.

Botones (FButton)

El componente de botón se presenta en cuatro variantes según el contexto de uso:

FButton — VARIANTES		
VARIANTE	MUESTRA	USO
Primary		Confirmaciones en formularios
Outline		Acciones secundarias
Text		Acciones en tablas y listados
Destruyctivo		Bajas y ceses

Ilustración 9: variantes del componente FButton. Fuente: elaboración propia.

- **Primary:** fondo índigo corporativo con texto blanco. Se usa en las confirmaciones de los formularios de alta y edición, y en los botones de acción principal del Dashboard.
- **Outline:** fondo transparente con borde y texto en índigo. Reservado para acciones secundarias como el acceso rápido a la gestión de empleados desde el Dashboard.
- **Text:** sin fondo ni borde, solo el texto en índigo. Se usa en acciones dentro de tablas y listados donde el botón no debe llamar la atención.
- **Destruyctivo:** fondo rojo muy tenue con texto en rojo. Comunica la naturaleza de riesgo de acciones como dar de baja a un empleado o cerrar un proyecto, sin resultar agresivo visualmente.

Campos de formulario (CrystallInput)

Los campos de formulario tienen fondo semitransparente con un borde muy sutil que apenas se percibe en reposo. Al recibir el foco, el borde cambia al índigo corporativo y aparece un halo suave alrededor del campo para indicar claramente cuál está activo. En estado de error, el borde pasa a rojo y aparece el mensaje de validación debajo del campo.

Las etiquetas se muestran en mayúsculas, en tamaño reducido y color gris medio, generando una jerarquía visual clara entre la etiqueta y el valor introducido. La altura mínima del campo es de 48 px para que la interacción táctil sea cómoda.

Ilustración 10: estados del campo de formulario CrystalInput. Fuente: elaboración propia.

Barra de búsqueda (EliteSearch)

La barra de búsqueda aparece en reposo de forma muy discreta, con fondo transparente y un borde inferior sutil, para no competir visualmente con el contenido de las tablas. Al recibir el foco, se expande horizontalmente, adquiere un fondo suave con efecto de desenfoque y el icono de la lupa cambia al índigo corporativo para indicar que está activa.

EMPLEADO ^	NIF / NIE ^	F. NACIMIENTO ^	TELÉFONO ^	EST. CIVIL ^	FORMACIÓN ^
Adriana María Costa Flores adriana.costa@omnia.com	42526272P	25/10/1998	603222333	Soltero	Si
Alberto Cano Ruiz alberto.cano@omnia.com	01234567J	14/8/1987	600000111	Casado	Si
Alejandro Cruz Herrero alejandro.cruz@omnia.com	20304050U	14/2/1988	601111222	Casado	Si

Ilustración 11: barra de búsqueda EliteSearch en reposo. Fuente: elaboración propia.

Alta Empleado

Q

Buscar empleado...

EMPLEADO	NIF / NIE	F. NACIMIENTO	TELÉFONO	EST. CIVIL	FORMACIÓN
Adriana María Costa Flores adriana.costa@omnia.com	42526272P	25/10/1998	603222333	Soltero	Si
Alberto Cano Ruiz alberto.cano@omnia.com	01234567J	14/8/1987	600000111	Casado	Si
Alejandro Cruz Herrero alejandro.cruz@omnia.com	20304050U	14/2/1988	601111222	Casado	Si

Ilustración 12: barra de búsqueda EliteSearch con foco. Fuente: elaboración propia.

Notificaciones (CrystalToast)

Las notificaciones aparecen centradas en la parte superior de la pantalla, superpuestas al contenido, y desaparecen automáticamente tras unos segundos. Siguen la misma estética que las tarjetas: fondo blanco semitransparente con efecto de desenfoque y esquinas redondeadas. El color del icono varía según el tipo de mensaje: índigo para confirmaciones de éxito, rojo para errores e información en gris.

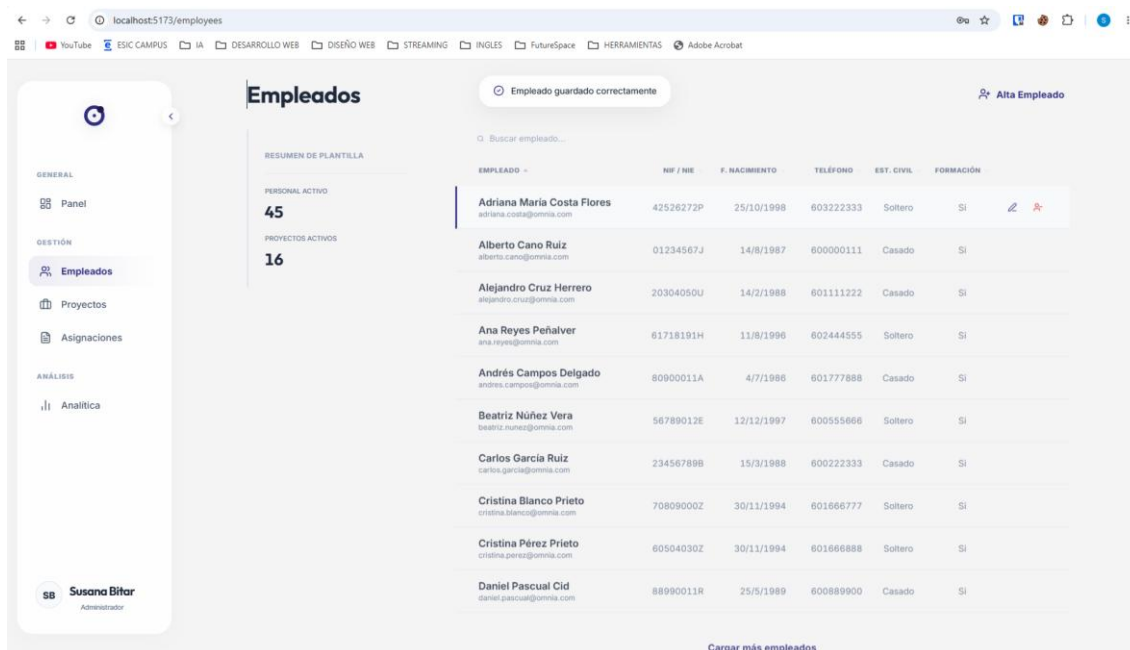


Ilustración 13: notificaciones CrystalToast de éxito. Fuente: elaboración propia.

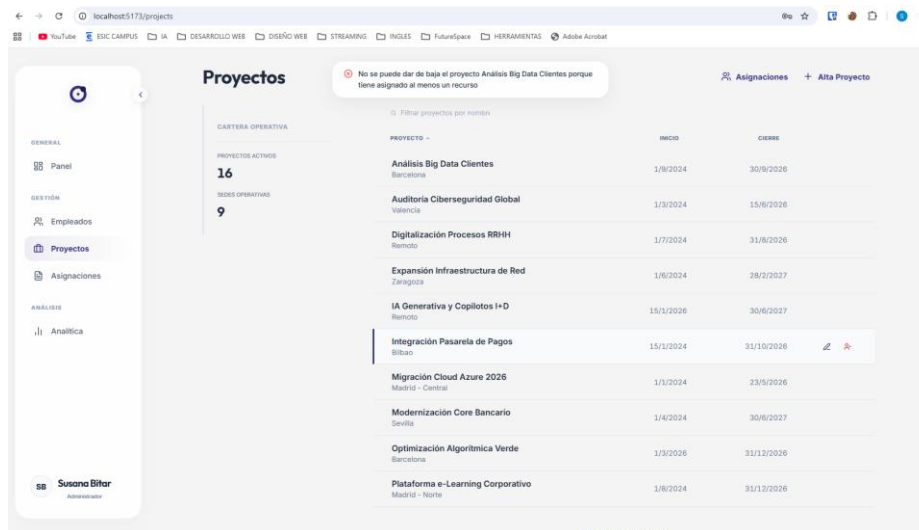


Ilustración 14: notificaciones CrystalToast de error. Fuente: elaboración propia.

Listas de datos

Las listas de empleados, proyectos y asignaciones siguen un diseño de tabla minimalista: filas separadas únicamente por un borde inferior sutil, sin fondos de celda ni separadores visuales pesados. Al pasar el cursor sobre una fila, aparece un acento lateral izquierdo en índigo que guía la vista y un fondo muy claro que identifica la fila activa sin resultar intrusivo.

Las columnas se distribuyen con anchos proporcionales adaptados al contenido de cada módulo: en empleados el nombre ocupa más espacio que los campos secundarios; en proyectos la descripción tiene mayor peso que la localización o las fechas.

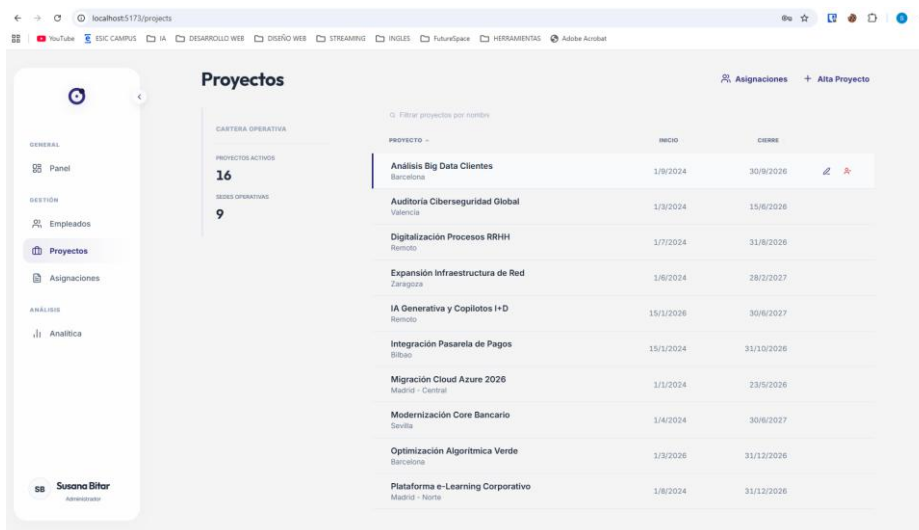


Ilustración 15: lista de proyectos con fila en hover. Fuente: elaboración propia.

5.2. Definición de componentes

A continuación, se describen los flujos técnicos principales que articulan el funcionamiento de la aplicación.

Login de usuario

El usuario introduce su correo y contraseña en LoginView.vue. Al enviar el formulario, el frontend realiza una petición POST /api/auth/login al backend. El AuthController recibe las credenciales y delega en AuthService, que localiza el usuario en la tabla US_USUARIOS por email y compara la contraseña recibida con el hash almacenado usando BCrypt. Si la comparación es válida, el servicio devuelve un objeto con los datos del usuario (sin contraseña). El frontend recibe la respuesta, guarda los datos en el store de Pinia y los persiste en localStorage. A partir de ese momento, Vue Router permite el acceso a las rutas privadas.

Alta de empleado

Desde EmployeeFormView.vue, el usuario rellena el formulario de nuevo empleado. Tras la validación en cliente, el frontend envía una petición POST /api/employees con el cuerpo en JSON. El EmployeeController recibe la petición y llama a EmployeeService.create(), que mapea el DTO recibido a la entidad Employee y la persiste mediante el repositorio JPA. El campo F_BAJA queda nulo, indicando que el empleado está activo. La respuesta incluye el empleado creado como DTO, sin exponer la entidad JPA directamente.

Baja lógica de empleado (soft delete)

Cuando el administrador confirma el cese de un trabajador, el frontend envía una petición DELETE /api/employees/{id}. El EmployeeService no elimina el registro físicamente: rellena el campo F_BAJA con la fecha actual y persiste el cambio. A partir de ese momento, las consultas de listado filtran por F_BAJA IS NULL, de modo que el empleado desaparece de la vista operativa pero el histórico queda intacto en base de datos.

Baja de proyecto con restricción de asignaciones

Al intentar dar de baja un proyecto, ProjectService comprueba si existen registros activos en la tabla PR_EMPLEADOS_PROYECTO con ese ID_PROYECTO. Si los hay, lanza una excepción que el controlador transforma en una respuesta HTTP 409, y el frontend muestra un mensaje de error explicando que el proyecto tiene personal asignado. Si no hay asignaciones activas, se rellena el

campo F_BAJA del proyecto con la fecha actual.

Asignación de empleado a proyecto

Desde AssignmentsView.vue, el usuario selecciona un proyecto y añade un empleado a su equipo. El frontend envía una petición POST /api/assignments con el ID del proyecto y el ID del empleado. El AssignmentService construye la clave primaria compuesta (EmployeeProjectId) y persiste el nuevo registro en PR_EMPLEADOS_PROYECTO con la fecha de alta actual. Para desasignar, se envía DELETE /api/assignments/{idProyecto}/{idEmpleado} y el servicio elimina el registro de la tabla intermedia.

Analítica

El módulo de Analítica realiza una única petición GET /api/analytics al backend, que devuelve todas las métricas del sistema en una sola respuesta. Esta decisión de diseño minimiza la latencia del cliente al evitar múltiples llamadas en cascada. Todos los cálculos (índice de rotación, carga por proyecto, distribución geográfica, evolución de contrataciones) se ejecutan en el servidor mediante consultas JPA agregadas, de modo que el navegador únicamente recibe los datos ya procesados. AnalyticsView.vue pasa esos datos a Chart.js, que renderiza los gráficos interactivos de anillo, barras y líneas.

5.3. Diseño de tablas de la base de datos

El modelo relacional de Omnia Manager se implementa en una base de datos MySQL denominada omnia, con codificación utf8mb4 y cotejamiento utf8mb4_unicode_ci para soporte completo de caracteres especiales, incluidos acentos y símbolos del español. El esquema está compuesto por cuatro tablas que se detallan a continuación.

Tabla US_USUARIOS

Almacena las cuentas de los administradores del sistema.

<i>Campo</i>	<i>Tipo</i>	<i>Restricciones</i>	<i>Descripción</i>
ID_USUARIO	INT	PK, AUTO_INCREMENT	Identificador único del usuario
TX_EMAIL	VARCHAR(100)	UK, NOT NULL	Correo corporativo. Identificador inmutable
TX_PASSWORD	VARCHAR(100)	NOT NULL	Contraseña cifrada con BCrypt

TX_ROLE	VARCHAR(20)	NOT NULL	Rol del usuario en el sistema
F_ALTA	DATE	NOT NULL	Fecha de creación de la cuenta
TX_NOMBRE	VARCHAR(100)	NULL	Nombre a mostrar en la interfaz

Tabla 1: Estructura de la tabla US_USUARIOS. Fuente: elaboración propia.

Tabla EM_EMPLEADOS

Almacena la ficha de cada trabajador de la empresa.

Campo	Tipo	Restricciones	Descripción
ID_EMPLEADO	INT	PK, AUTO_INCREMENT	Identificador único del empleado
TX_NIF	VARCHAR(9)	UK, NOT NULL	NIF del trabajador (9 caracteres)
TX_NOMBRE	VARCHAR(30)	NOT NULL	Nombre
TX_APELLIDO1	VARCHAR(40)	NOT NULL	Primer apellido
TX_APELLIDO2	VARCHAR(40)	NOT NULL	Segundo apellido
F_NACIMIENTO	DATE	NOT NULL	Fecha de nacimiento
N_TELEFONO1	VARCHAR(12)	NOT NULL	Teléfono principal
N_TELEFONO2	VARCHAR(12)	NOT NULL	Teléfono secundario
TX_EMAIL	VARCHAR(40)	NOT NULL	Correo electrónico del trabajador
F_ALTA	DATE	NOT NULL	Fecha de incorporación a la empresa
F_BAJA	DATE	NULL	Fecha de cese. Nulo si el empleado está activo
CX_EDOCIVIL	VARCHAR(1)	NOT NULL	Estado civil: S soltero, C casado
B_FORMACIONU	VARCHAR(1)	NOT NULL	Formación universitaria: S sí, N no

Tabla 2: Estructura de la tabla EM_EMPLEADOS. Fuente: elaboración propia.

Tabla PR_PROYECTOS

Almacena los proyectos de la compañía.

<i>Campo</i>	<i>Tipo</i>	<i>Restricciones</i>	<i>Descripción</i>
ID_PROYECTO	INT	PK, AUTO_INCREMENT	Identificador único del proyecto
TX_DESCRIPCION	VARCHAR(125)	NOT NULL	Descripción del proyecto
F_INICIO	DATE	NOT NULL	Fecha de inicio
F_FIN	DATE	NULL	Fecha prevista de finalización
F_BAJA	DATE	NULL	Fecha de cierre lógico. Nulo si el proyecto está activo
TX_LUGAR	VARCHAR(30)	NOT NULL	Sede o localización
TX_OBSERVACIONES	VARCHAR(300)	NULL	Notas adicionales

Tabla 3: Estructura de la tabla PR_PROYECTOS. Fuente: elaboración propia.

Tabla PR_EMPLEADOS_PROYECTO

Tabla de unión que resuelve la relación muchos a muchos entre empleados y proyectos.

<i>Campo</i>	<i>Tipo</i>	<i>Restricciones</i>	<i>Descripción</i>
ID_PROYECTO	INT	PK, FK → PR_PROYECTOS	Parte de la clave primaria compuesta
ID_EMPLEADO	INT	PK, FK → EM_EMPLEADOS	Parte de la clave primaria compuesta
F_ALTA	DATE	NOT NULL	Fecha de asignación del empleado al proyecto

Tabla 4: Estructura de la tabla PR_EMPLEADOS_PROYECTO. Fuente: elaboración propia.

6. DEMOSTRACIÓN

El código fuente completo está disponible en el siguiente repositorio:

<https://github.com/Susybit/omnia-manager>

Requisitos previos

Para ejecutar la aplicación es necesario tener instalado Docker Desktop en el equipo. No se requiere ninguna otra dependencia adicional.

Instalación, arranque y parada

1. Descomprimir el fichero del proyecto o clonar el repositorio.
2. Abrir una terminal en la raíz del proyecto, donde se encuentra el fichero docker-compose.yml.
3. Para arrancar la aplicación ejecutar: **docker compose up --build**

Docker descargará las imágenes necesarias, construirá los contenedores de base de datos, backend y frontend, y los pondrá en marcha automáticamente. El proceso puede tardar unos minutos en la primera ejecución.

4. Para detenerlos, ejecutar **docker compose down**.

Acceso a la aplicación

Una vez arrancados los tres contenedores, abrir el navegador y acceder a: <http://localhost:5173/>

Credenciales de acceso

La base de datos se inicializa automáticamente con datos de prueba. Se pueden utilizar las siguientes cuentas de administrador:

Usuario	Contraseña
<i>admin@omnia.com</i>	S12345678*
susana@omnia.com	S12345678*

Tabla 5: Credenciales de acceso para la demo. Fuente: elaboración propia.

7. CONCLUSIONES

El desarrollo de Omnia Manager ha supuesto completar, por primera vez de forma autónoma, un proyecto full-stack completo: desde el diseño de la base de datos hasta el despliegue en contenedores Docker. Todos los requisitos funcionales definidos al inicio del proyecto se han implementado: gestión de empleados y proyectos con baja lógica, asignaciones, analítica y autenticación segura con BCrypt.

A nivel técnico, el mayor aprendizaje ha sido la coordinación entre las tres capas de la aplicación y la contenerización del entorno completo con Docker Compose, lo que garantiza que la aplicación funcione de forma idéntica en cualquier máquina sin configuración adicional.

Como líneas de mejora futuras, se podría incorporar un sistema de autenticación basado en tokens, la gestión de múltiples roles de usuario, y una versión adaptada para dispositivos móviles. Una evolución especialmente relevante sería la implementación de perfiles de empresa, de forma que la plataforma pudiera ser utilizada por distintas organizaciones de manera independiente, cada una gestionando sus propios empleados, proyectos y usuarios desde un espacio aislado. No obstante, el resultado obtenido cumple con los objetivos planteados y sienta una base sólida sobre la que seguir construyendo.

8. BIBLIOGRAFÍA

Axios Contributors. (2024). *Axios — Promise based HTTP client for the browser and Node.js*.

Axios. Último acceso de 14 de marzo de 2026 en: <https://axios-http.com/docs/intro>

Docker, Inc. (2024). *Docker Documentation. Docker Docs*. Último acceso de 7 de marzo de 2026

en: <https://docs.docker.com>

F5 Networks. (2024). *NGINX Documentation. NGINX*. Último acceso de 19 de abril de 2026 en:

<https://nginx.org/en/docs/>

Learn Git Branching. (s.f.). *Learn Git Branching — Aprende Git de forma interactiva. Learn Git Branching*. Último acceso de 3 de marzo de 2026 en:

https://learngitbranching.js.org/?locale=es_ES

Microsoft. (2024). *Tutorial de Docker para Visual Studio. Microsoft Learn*. Último acceso de 11

de marzo de 2026 en: <https://learn.microsoft.com/es-es/visualstudio/docker/tutorials/docker-tutorial>

Oracle Corporation. (2024). *MySQL 8.0 Reference Manual. MySQL*. Último acceso de 23 de abril

de 2026 en: <https://dev.mysql.com/doc/refman/8.0/en/>

Posva, E. (2024). *Pinia — The intuitive store for Vue.js. Pinia*. Último acceso de 26 de marzo de

2026 en: <https://pinia.vuejs.org>

Spring Team. (2024). *Spring Boot Reference Documentation. Spring*. Último acceso de 5 de abril

de 2026 en: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring Team. (2024). *Spring Data JPA Tutorial. YouTube*. Último acceso de 17 de marzo de 2026

en: https://www.youtube.com/watch?v=8SGI_XS5OPw

Udemy. (2025). *Git Started with GitHub. Udemy*. Último acceso de 9 de marzo de 2026 en:

<https://www.udemy.com/course/git-started-with-github/>

Udemy. (2025). *Programación orientada a objetos y principios SOLID. Udemy*. Último acceso de

28 de marzo de 2026 en: <https://www.udemy.com/course/programacion-orientada-a-objetos-y-principio-solid/>

Udemy. (2025). *Spring Framework and Dependency Injection for Beginner*. Udemy. Último acceso de 6 de abril de 2026 en: <https://www.udemy.com/course/restful-web-services-with-spring-framework-a-quick-start>

Udemy. (2025). *Vue 3 JS — Iniciante en este framework*. Udemy. Último acceso de 15 de abril de 2026 en: <https://www.udemy.com/course/vue-3-js-iniciate-en-este-framework/>

Vue.js Team. (2024). *Vue Router — The official router for Vue.js*. Vue Router. Último acceso de 30 de marzo de 2026 en: <https://router.vuejs.org>

Vue.js Team. (2024). *Vue.js — The Progressive JavaScript Framework*. Vue.js. Último acceso de 22 de abril de 2026 en: <https://vuejs.org>

Vue.js Team. (s.f.). *Vue.js Style Guide — Guía de buenas prácticas*. Vue.js. Último acceso de 8 de abril de 2026 en: <https://es.vuejs.org/v2/style-guide/>

Vuetify LLC. (2024). *Vuetify — Vue Component Framework*. Vuetify. Último acceso de 1 de abril de 2026 en: <https://vuetifyjs.com>

xurxodev. (s.f.). *Refactorizar código — Guía práctica de Clean Code*. xurxodev. Último acceso de 21 de marzo de 2026 en: <https://xurxodev.com/refactorizar-codigo/>

9. ANEXOS

9.1. Código fuente

El código fuente completo está disponible en el repositorio <https://github.com/Susybit/omnia-manager> y entregado en formato ZIP en Canvas. A continuación, se muestra la estructura del proyecto y una descripción de los ficheros principales.

<i>Fichero</i>	<i>Descripción</i>
<i>docker-compose.yml</i>	Orquesta los tres contenedores (base de datos, backend y frontend) en un único entorno
<i>frontend/Dockerfile</i>	Construcción multietapa: compila la SPA con Node 22 y la sirve con Nginx Alpine
<i>backend/Dockerfile</i>	Empaqueta el JAR de Spring Boot en una imagen Java 17 Alpine
<i>mysql_DDL_OMNIA.sql</i>	DDL con la definición de las cuatro tablas del esquema omnia
<i>data.sql</i>	Dataset de prueba con 2 usuarios, 55 empleados, 20 proyectos y 96 asignaciones
<i>application.yml</i>	Configuración de base de datos, puerto y comportamiento de Spring Boot
<i>BackendApplication.java</i>	Punto de entrada de la aplicación Spring Boot
<i>SecurityConfig.java</i>	Configura CORS y deshabilita CSRF para la API REST
<i>AuthController.java</i>	Endpoint de login: valida credenciales y devuelve los datos del usuario
<i>EmployeeController.java</i>	CRUD de empleados con baja lógica (soft delete)
<i>ProjectController.java</i>	CRUD de proyectos con restricción de asignaciones activas
<i>AnalyticsController.java</i>	Endpoint único GET /analytics que devuelve todas las métricas del sistema
<i>main.js</i>	Punto de entrada de Vue: inicializa Vuetify, Pinia y Vue Router
<i>router/index.js</i>	Define rutas públicas y privadas con guardia de navegación
<i>stores/auth.js</i>	Store de Pinia que gestiona la sesión del administrador
<i>EmployeesView.vue</i>	Vista principal de gestión de empleados
<i>ProjectsView.vue</i>	Vista principal de gestión de proyectos
<i>AssignmentsView.vue</i>	Vista de asignación de empleados a proyectos
<i>AnalyticsView.vue</i>	Módulo de analítica con gráficos interactivos Chart.js
<i>FButton.vue</i>	Componente de botón con variantes solid, outline, text y destructivo

Tabla 6: Ficheros principales del proyecto Omnia Manager. Fuente: elaboración propia.